



Figure 5: Differences between the assembly codes generated with and without the 'volatile' keyword

```

14      //Some code goes here
15 }
16
17 void thread_two(void)
18 {
19     //some code goes here
20
21     global_item_count++;
22     sleep(5);
23
24     //some code goes here
25
26 }

```

In the above demo program, the compiler doesn't have any knowledge of the context switching between two threads. If the compiler optimisations are turned 'ON', then the compiler will assume that the *global_item_count* variable is always 'ZERO' and no other part of the thread is attempting to modify it. So, the compiler may replace the *while* loop in the code above, as shown in the code below:

```

.
.
while(TRUE)
{
    sleep(1);
}
.
.

```

...which is nothing but the infinite loop; so in order to avoid such optimisations by the compiler, it is safe to declare the variable *global_item_count* as 'volatile'. Similarly, one can realise the effect of the producer-consumer problem accessing the global variable without declaring it as 'volatile'.

Illustration 4: Interrupt service routines

Let us consider another example given in the code snippet below, where 'volatile' plays a very important role in the ISR (Interrupt Service Routines):

```

1 int flag = 0;
2 void rx_isr(void)
3 {
4     flag = 1;
5 }
6 int main()
7 {
8     // ...
9     while(!flag)
10    {
11        //Some code goes here
12    }
13    // ...
14 }

```

In the above example, if the flag is not declared as 'volatile', the compiler may optimise the code (assuming always that the flag is ZERO) and replace the *while(!flag)* to *while(TRUE)*, which is nothing but the infinite loop. But the flag value might change when the interrupt occurs.

 **Note :** Whether to declare the variable as 'volatile' or not is cross-compiler dependent. Anyhow it is a good practice to declare the variable as 'volatile' to achieve the portability of the code.

A variable should be declared volatile whenever its value can change asynchronously. In real time, three types of variables can change:

- Memory-mapped peripheral registers (e.g., polling and waiting)
- Global variables modified by an Interrupt Service Routine
- Global variables accessed by multiple tasks within a multi-threaded application

The main use of the 'volatile' key word is to prevent the compiler from optimising the code in terms of time complexity, by generating a code that uses CPU registers as faster ways to represent variables. Declaring the variable as 'volatile' forces compiled code to access the exact memory location in RAM on every access to the variable to get its latest value, thereby avoiding any runtime surprises for the programmer. 

By: Satyanarayana Sampangi

The author is a member of the embedded software team at Emertxe Information Technology (P) Ltd (<http://www.emertxe.com>). His areas of interest are embedded C programming combined with data structures and microcontrollers. He can be reached at satya@emertxe.com